

```

1  """prelude.py -- Haskell-style Prelude in pure Python, in pedagogical order.
2
3  Pascal discipline: strict define-before-use. Read top to bottom and every name is already defined
4  when you meet it. Sections build on each other:
5
6      1 combinators          2 pairs          3 arithmetic
7      4 list basics         5 higher-order lists  6 folds
8      7 scans               8 streams (lazy)   9 slicing & spans
9      10 sorting & searching 11 strings       12 containers
10     13 nodes              14 grids & windows  15 control
11
12  Within a section names run alphabetically, except that a definition is hoisted ahead of the first
13  name that uses it (foldl before compose, span before break_), so the discipline holds inside
14  sections too.
15
16  Import qualified (import prelude as P): Haskell names shadow builtins by design. Beyond the Prelude
17  proper: the working parts of Data.List, Data.Map, Data.Maybe, Data.Function and friends are folded
18  into the same sections -- grouped by what they do, not by the module they came from. None is
19  Nothing throughout. foldr = foldl(flip(f), reversed(xs), init): iterative, stack-safe, valid only
20  because Python is strict.
21  """
22
23  # ===== 1. combinators ===== (functions about functions)
24
25  const      = lambda x: lambda _: x
26  curry      = lambda f: lambda x: lambda y: f(x, y)
27  flip       = lambda f: (lambda x, y: f(y, x))      # flip f x y = f y x
28  fromMaybe = lambda d, x: d if x is None else x     # Data.Maybe: default for every None-returning tool
29  id         = lambda x: x                          # shadows builtin id() by design
30  isJust     = lambda x: x is not None               # Data.Maybe: None test as a handable predicate
31  isNothing  = lambda x: x is None
32  NOTHING    = object()                            # Maybe's Nothing: "no arg given"; test with 'is'
33  on         = lambda f, g: lambda x, y: f(g(x), g(y)) # Data.Function: combine via a key -- cmp 'on' fst
34  partial    = lambda f, *bound: lambda *args: f(*bound, *args)
35  uncurry    = lambda f: lambda p: f(*p)
36
37  # ===== 2. pairs =====
38
39  fst = lambda p: p[0]
40  snd = lambda p: p[1]
41  swap = lambda p: (p[1], p[0])    # Data.Tuple
42
43  # ===== 3. arithmetic & logic =====
44
45  div      = lambda a, b: a // b                # floors, like Haskell div
46  even     = lambda n: n % 2 == 0
47  gcd      = lambda a, b: abs(a) if b == 0 else gcd(b, a % b)    # >= 0 like Haskell; depth <= 90
48  hypot    = lambda x, y: (x*x + y*y) ** 0.5
49
50  def isqrt(n): # floor sqrt without floats (Newton)
51      if n < 0:
52          raise ValueError("isqrt of negative")
53      x = n
54      y = (x + 1) // 2
55      while y < x:
56          x, y = y, (y + n // y) // 2
57      return x if n else 0
58
59  lcm      = lambda a, b: abs(a // gcd(a, b) * b) if a and b else 0    # >= 0 like Haskell
60  mod      = lambda a, b: a % b
61  not_     = lambda b: not b      # 'not' is a Python keyword
62  odd      = lambda n: n % 2 == 1
63  otherwise = True
64  pred     = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1 # Enum: numbers and Chars
65  quot     = lambda a, b: -(a // b) if (a < 0) != (b < 0) else a // b    # truncates, like Haskell quot
66  rem      = lambda a, b: a - b * quot(a, b)
67  quotRem  = lambda a, b: (quot(a, b), rem(a, b))
68  signum   = lambda x: (x > 0) - (x < 0)
69  succ     = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1
70
71  # ===== 4. list basics =====
72
73  drop      = lambda n, xs: list(xs)[max(n, 0):]    # finite lists; n<0 drops none
74  elem      = lambda x, xs: x in xs
75  zip_      = lambda a, b: list(zip(a, b))          # shortest wins, any iterable
76  enum      = lambda xs, start=0: zip_(list(range(start, start + len(xs))), xs)
77  head      = lambda xs: xs[0]
78  init      = lambda xs: xs[:-1]
79  isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p)    # Data.List; strings or lists
80  isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p) # not [-len(p):]: [-0:] is ALL
81  last      = lambda xs: xs[-1]
82  lookup    = lambda k, pairs: next((v for kk, v in pairs if kk == k), None) # Nothing -> None
83  notElem   = lambda x, xs: x not in xs
84  nub       = lambda xs: list(dict.fromkeys(xs))        # unique; first seen wins
85  null      = lambda xs: len(xs) == 0
86  pairwise  = lambda xs: list(zip(xs, xs[1:]))          # zip xs (tail xs)
87  replicate = lambda n, x: [x] * n
88  splitAt   = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs) # n<0 splits at the front

```

```

89 stripPrefix = lambda p, xs: xs[len(p):] if isPrefixOf(p, xs) else None      # Just the rest, or Nothing
90 tail        = lambda xs: xs[1:]
91 def transpose(xss):
92     # Data.List: rows <-> cols, RAGGED-SAFE like Haskell
93     xss = [list(xs) for xs in xss]    # any iterables in (rows may be one-shot); materialise once
94     n = max(map(len, xss), default=0) # short rows just drop out of later columns (no truncation)
95     return [[xs[i] for xs in xss if i < len(xs)] for i in range(n)]
96 unzip       = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], [])
97 zip3        = lambda a, b, c: list(zip(a, b, c))
98 # ===== 5. higher-order lists =====
99
100 catMaybes = lambda xs: [x for x in xs if x is not None]    # mapMaybe id
101 concat    = lambda xss: [x for xs in xss for x in xs]
102 concatMap = lambda f, xs: [y for x in xs for y in f(x)]
103 cross     = lambda a, b: [(x, y) for x in a for y in b]    # cartesian; 'product' is the fold
104 filter_   = lambda pred, xs: [x for x in xs if pred(x)]
105 # Data.List find: lazy first-match -- folds can't stop early, find can
106 find      = lambda pred, xs: next((x for x in xs if pred(x)), None) # first match, else None
107 map_      = lambda f, xs: [f(x) for x in xs]
108 # Data.Maybe mapMaybe: map and drop the Nothings in ONE pass -- the parse-and-filter shape
109 mapMaybe = lambda f, xs: [y for y in (f(x) for x in xs) if y is not None]
110 def partition(pred, xs):
111     # (keepers, rest) in ONE pass; pred called once per element
112     yes, no = [], []
113     for x in xs:
114         (yes if pred(x) else no).append(x)
115     return (yes, no)
116 starmap   = lambda f, pairs: [f(*p) for p in pairs]        # map . uncurry
117 zipWith   = lambda f, a, b: [f(x, y) for x, y in zip(a, b)]
118 zipWith3  = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)]
119 # ===== 6. folds ===== (collapse a list to one value)
120
121 def foldl(f, xs, init=NOTHING): # THE left fold; Python buried its own in functools as reduce
122     it = iter(xs)
123     if init is NOTHING:
124         try:
125             acc = next(it)
126         except StopIteration:
127             raise TypeError("fold of empty sequence with no initial value")
128     else:
129         acc = init
130     for x in it:
131         acc = f(acc, x)
132     return acc
133
134 foldr     = lambda f, xs, init: foldl(flip(f), reversed(list(xs)), init)
135 compose  = lambda *fns: lambda x: foldr(lambda f, acc: f(acc), fns, x) # foldr (.) id: RIGHTMOST first
136 foldl1   = lambda f, xs: foldl(f, xs)
137 foldr1   = lambda f, xs: foldl(flip(f), reversed(list(xs)))
138 product  = lambda xs: foldl(lambda a, x: a * x, xs, 1)        # the numeric fold
139
140 # enumeration folds -- brute-force licenses for small n (say the bound out loud):
141 # Control.Monad replicateM: every length-n word over xs -- cross, n times; |xs|^n
142 replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]])
143 # Data.List subsequences: the powerset; each element DOUBLES acc -- 2^n, fine to n ~ 20
144 subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]])
145
146 # ===== 7. scans ===== (a fold that keeps its history)
147
148 def accumulate(xs, f=None, initial=NOTHING): # scanl / scanl1
149     if f is None:
150         f = lambda a, b: a + b
151     it = iter(xs)
152     if initial is NOTHING:
153         try:
154             acc = next(it)
155         except StopIteration:
156             return
157     else:
158         acc = initial
159     yield acc
160     for x in it:
161         acc = f(acc, x)
162         yield acc
163
164 scanl     = lambda f, z, xs: list(accumulate(xs, f, initial=z))
165 scanl1    = lambda f, xs: list(accumulate(xs, f))
166 scanr     = lambda f, z, xs: list(reversed(scanl(flip(f), z, list(reversed(list(xs)))))
167 scanr1    = lambda f, xs: list(reversed(scanl1(flip(f), list(reversed(list(xs)))))
168
169 # ===== 8. streams ===== (lazy, possibly infinite)
170
171 chain     = lambda *iterables: (x for it in iterables for x in it) # concat, lazily
172
173 def count(start=0, step=1):
174     n = start
175     while True:
176         yield n

```

```

177         n += step
178
179 def cycle(xs):
180     xs = list(xs)
181     if not xs:
182         raise ValueError("cycle: empty list")    # Haskell errors too; the alternative is a silent hang
183     while True:
184         yield from xs
185
186 def iterate(f, x):
187     while True:
188         yield x
189         x = f(x)
190
191 def repeat(x, n=None):
192     if n is None:
193         while True:
194             yield x
195     else:
196         for _ in range(n):
197             yield x
198
199 def unfoldr(f, seed):
200     out = []
201     step = f(seed)
202     while step is not None:
203         val, seed = step
204         out.append(val)
205         step = f(seed)
206     return out
207
208 # ===== 9. slicing & spans ===== (finite views of streams)
209
210 def span(p, xs):
211     xs = list(xs)
212     i = next((i for i, x in enumerate(xs) if not p(x)), len(xs))
213     return (xs[:i], xs[i:])
214 break_ = lambda p, xs: span(lambda x: not p(x), xs)
215 chunksOf = lambda n, xs: [xs[i:i + n] for i in range(0, len(xs), n)]    # Data.List.Split; short last ok
216
217 def dropwhile(pred, xs):
218     it = iter(xs)
219     for x in it:
220         if not pred(x):
221             yield x
222             break
223     yield from it
224
225 dropWhile = lambda p, xs: list(dropwhile(p, xs))
226 inits = lambda xs: [xs[:i] for i in range(len(xs) + 1)]    # Data.List: every prefix, [] first
227
228 def islice(iterable, stop):
229     it = iter(iterable)
230     for _ in range(stop):
231         try:
232             yield next(it)
233         except StopIteration:
234             return
235
236 tails = lambda xs: [xs[i:] for i in range(len(xs) + 1)]    # every suffix; substrings start here
237 take = lambda n, xs: list(islice(iter(xs), n))    # works on infinite streams
238
239 def takeuntil(pred, xs):
240     for x in xs:
241         yield x
242         if pred(x):
243             return
244
245 def takewhile(pred, xs):
246     for x in xs:
247         if not pred(x):
248             return
249         yield x
250
251 takeWhile = lambda p, xs: list(takewhile(p, xs))
252
253 # ===== 10. sorting & searching =====
254
255 def bisect_left(a, x):
256     lo, hi = 0, len(a)
257     while lo < hi:
258         mid = (lo + hi) // 2
259         if a[mid] < x:
260             lo = mid + 1
261         else:
262             hi = mid
263     return lo
264

```

```

265 def bisect_right(a, x):                                # first index where a[i] > x
266     lo, hi = 0, len(a)
267     while lo < hi:
268         mid = (lo + hi) // 2
269         if a[mid] <= x:
270             lo = mid + 1
271         else:
272             hi = mid
273     return lo
274
275 def cmp_to_key(cmp):                                    # resurrect Python 2's cmp for sortBy
276     class K:
277         def __init__(self, x): self.x = x
278         def __lt__(self, other): return cmp(self.x, other.x) < 0
279     return K
280
281 maxOn = lambda f, xs: max(xs, key=f)                    # maximumBy (comparing f) in O(n); first wins ties
282
283 def merge(a, b):                                        # merge two sorted lists, stable; the heart of merge sort
284     out, i, j = [], 0, 0                                # (section 13's merge_lists is this same loop on ListNodes)
285     while i < len(a) and j < len(b):
286         if a[i] <= b[j]:
287             out.append(a[i]); i += 1
288         else:
289             out.append(b[j]); j += 1
290     return out + a[i:] + b[j:]
291
292 minOn = lambda f, xs: min(xs, key=f)                    # minimumBy (comparing f): sortOn + head without the sort
293 sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp)) # comparator sort, Python 2 style
294 sortOn = lambda f, xs: sorted(xs, key=f)                # sortOn (schwartzian, f called once per element)
295
296 # ===== 11. strings =====
297
298 lines = str.splitlines
299 unlines = lambda ls: "".join(l + "\n" for l in ls)
300 unwords = " ".join
301 words = str.split
302
303 # ===== 12. containers =====
304
305 # multiset ops on Counters (use .get, not [] -- indexing a Counter autovivifies zeros):
306 bag_diff = lambda a, b: {k: a[k] - b[k] for k in a if a[k] > b.get(k, 0)} # clipped at 0
307 bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a.get(k, 0), b.get(k, 0)))}
308 bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items()) # a <= b (inclusion)
309 bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
310
311 class defaultdict(dict):
312     def __init__(self, factory=None, *args, **kw):
313         super().__init__(*args, **kw)
314         self.factory = factory
315     def __missing__(self, key):
316         if self.factory is None:
317             raise KeyError(key)
318         self[key] = self.factory()
319         return self[key]
320
321 def Counter(xs):                                        # count-by-value; a defaultdict(int) fold
322     d = defaultdict(int)
323     for x in xs:
324         d[x] += 1
325     return d
326
327 class deque:                                           # two-stack; all four ends amortized O(1)
328     def __init__(self, xs=()):
329         self._in, self._out = [], list(reversed(list(xs)))
330     def append(self, x): self._in.append(x)
331     def appendleft(self, x): self._out.append(x)
332     def pop(self):
333         if not self._in:
334             self._in, self._out = list(reversed(self._out)), []
335         return self._in.pop()
336     def popleft(self):
337         if not self._out:
338             self._out, self._in = list(reversed(self._in)), []
339         return self._out.pop()
340     def __len__(self): return len(self._in) + len(self._out)
341     def __iter__(self): return iter(self._out[::-1] + self._in)
342
343 def dsu(n):                                            # union-find, path halving; union -> False if already joined
344     parent = list(range(n))
345     def find(x):
346         while parent[x] != x:
347             parent[x] = parent[parent[x]]
348             x = parent[x]
349         return x
350     def union(a, b):
351         ra, rb = find(a), find(b)
352         if ra == rb:

```

```

353         return False
354     parent[ra] = rb
355     return True
356     return find, union
357
358 def fromListWith(f, pairs):          # Data.Map fromListWith: THE dict-building fold; f(new, old)
359     d = {}                          # like insertWith -- so grouping with concat REVERSES each
360     for k, v in pairs:              # group (the classic Haskell gotcha); use add for counts
361         d[k] = f(v, d[k]) if k in d else v
362     return d                        # Counter == fromListWith(add) over (x, 1);
363                                     # grouping == fromListWith(++) over (k, [v])
364
365 def getpath(t, ks, default=None):    # read along ks WITHOUT autovivifying
366     for k in ks:
367         if not isinstance(t, dict) or k not in t:
368             return default
369         t = t[k]
370     return t
371
372 def groupBy(eq, xs):                # Data.List groupBy: runs of CONSECUTIVE elements; each new
373     out = []                        # element is compared via eq against the run's FIRST element,
374     for x in xs:                    # exactly like Haskell (span-based) -- not its neighbor
375         if out and eq(out[-1][0], x):
376             out[-1].append(x)
377         else:
378             out.append([x])
379     return out
380
381 group = lambda xs: groupBy(lambda a, b: a == b, xs)    # Data.List group: runs of equals, [[a]] -- no keys
382
383 def heappop(h):                          # min-heap on a plain list: sift-down
384     h[0], h[-1] = h[-1], h[0]
385     top = h.pop()
386     i, n = 0, len(h)
387     while True:
388         s, l, r = i, 2*i + 1, 2*i + 2
389         if l < n and h[l] < h[s]: s = l
390         if r < n and h[r] < h[s]: s = r
391         if s == i:
392             return top
393         h[i], h[s] = h[s], h[i]
394         i = s
395
396 def heappush(h, x):                      # min-heap on a plain list: sift-up
397     h.append(x)
398     i = len(h) - 1
399     while i and h[(i - 1) // 2] > h[i]:
400         h[(i - 1) // 2], h[i] = h[i], h[(i - 1) // 2]
401         i = (i - 1) // 2
402
403 insertWith = lambda f, k, v, d: {**d, k: f(v, d[k]) if k in d else v}    # Data.Map insertWith: PERSISTENT
404                                     # (fresh dict); f(new, old)
405 ITree = lambda: defaultdict(ITree)
406
407 def paths(t):                            # cata for Tree/ITree: [(keypath, leaf)]
408     # recursion depth = tree height (word length / len(nums)) -- well under the ~1000 limit
409     if not isinstance(t, dict) or not t:    # leaf = non-dict value, or empty node
410         return [([], t)]
411     return [( [k] + p, leaf) for k, sub in t.items() for p, leaf in paths(sub)]
412
413 leaves = lambda t: [l for _, l in paths(t)]
414
415 def setpath(t, ks, v):                    # write leaf v at key path ks (autovivifies interior)
416     for k in ks[:-1]:
417         t = t[k]
418     t[ks[-1]] = v                        # NB: clobbers any subtree already at ks
419
420 Tree = lambda depth, leaf: (defaultdict(leaf) if depth == 1
421                             else defaultdict(lambda: Tree(depth - 1, leaf)))
422
423 unionWith = lambda f, a, b: {**a, **{k: f(a[k], v) if k in a else v for k, v in b.items()}}
424     # Data.Map unionWith: f(a's value, b's value) on shared keys;
425     # bag_union == unionWith(max), merge-sum == unionWith(add)
426
427 # ===== 13. nodes ===== (the LeetCode givens: binary trees & linked lists)
428
429 class ListNode:                          # the LeetCode linked list
430     def __init__(self, val=0, next=None):
431         self.val, self.next = val, next
432
433 class TreeNode:                          # the LeetCode binary tree
434     def __init__(self, val=0, left=None, right=None):
435         self.val, self.left, self.right = val, left, right
436
437 from_list = lambda xs: foldr(ListNode, xs, None)    # build; foldr of ListNode
438
439 def has_cycle(node):                      # Floyd: fast laps slow iff a cycle exists
440     slow = fast = node

```

```

441     while fast and fast.next:
442         slow, fast = slow.next, fast.next.next
443         if slow is fast:
444             return True
445     return False
446
447 # tfold: the tree cata -- f(val, l_acc, r_acc), z for the empty tree; recursion depth = tree height
448 tfold = lambda f, z, t: z if t is None else f(t.val, tfold(f, z, t.left), tfold(f, z, t.right))
449 inorder = lambda t: tfold(lambda v, l, r: l + [v] + r, [], t)
450
451 def levelorder(t):                                # BFS: the deque earning its keep
452     out, q = [], deque([t] if t else [])
453     while len(q):
454         n = q.popleft()
455         out.append(n.val)
456         if n.left: q.append(n.left)
457         if n.right: q.append(n.right)
458     return out
459
460 def merge_lists(a, b):                            # merge two sorted lists; dummy-head idiom
461     dummy = tail_ = ListNode()
462     while a and b:
463         if a.val <= b.val:
464             tail_.next, a = a, a.next
465         else:
466             tail_.next, b = b, b.next
467         tail_ = tail_.next
468     tail_.next = a or b
469     return dummy.next
470
471 def middle(node):                                  # fast/slow: second middle for even lengths
472     slow = fast = node
473     while fast and fast.next:
474         slow, fast = slow.next, fast.next.next
475     return slow
476
477 postorder = lambda t: tfold(lambda v, l, r: l + r + [v], [], t)
478 preorder = lambda t: tfold(lambda v, l, r: [v] + l + r, [], t)
479
480 def reverse_list(node):                            # three-pointer; prev is a fold accumulator
481     prev = None
482     while node:
483         node.next, prev, node = prev, node, node.next
484     return prev
485
486 tdepth = lambda t: tfold(lambda _, l, r: l + max(l, r), 0, t)
487 to_list = lambda node: list(unfoldr(lambda n: None if n is None else (n.val, n.next), node))
488 tsize = lambda t: tfold(lambda _, l, r: l + l + r, 0, t)
489
490 # ===== 14. grids & windows =====
491
492 def longest_window(xs, valid, add, rem):            # sliding-window skeleton; state lives in the closures
493     lo = best = 0
494     for hi in range(len(xs)):
495         add(xs[hi])
496         while not valid():                          # shrink until legal again
497             rem(xs[lo])
498             lo += 1
499         best = max(best, hi - lo + 1)
500     return best
501
502 def neighbors4(r, c, R, C):                        # in-bounds orthogonal neighbors
503     for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
504         if 0 <= r + dr < R and 0 <= c + dc < C:
505             yield r + dr, c + dc
506
507 def neighbors8(r, c, R, C):                        # ... plus diagonals
508     for dr in (-1, 0, 1):
509         for dc in (-1, 0, 1):
510             if (dr or dc) and 0 <= r + dr < R and 0 <= c + dc < C:
511                 yield r + dr, c + dc
512
513 # ===== 15. control =====
514
515 def memo(f):                                       # unbounded memoizer; enough for DP (no eviction by design)
516     cache = {}
517     def wrapped(*args, **kw):
518         key = (args, frozenset(kw.items())) if kw else args
519         if key not in cache:
520             cache[key] = f(*args, **kw)
521         return cache[key]
522     wrapped.cache = cache                          # peek at the DP table if curious
523     return wrapped
524
525 def until(p, f, x):                               # until p f x ? loop, not recursion (unbounded depth)
526     while not p(x):
527         x = f(x)
528     return x
end

```